SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Components of a computer system, including CPU, memory, and I/O. Basic Operational Concepts: Instruction execution cycle, instruction fetch and decode.Bus Structures: Single-bus, multi-bus, and hierarchical buses.Factors of Performance: CPU execution time, CPI, clock cycles, and benchmarking.8085 and 8086 Architectures: Overview, instruction sets, and addressing modes. Modern Architectures: ARM Cortex, Intel Core, and AMD Ryzen.Instruction Sets, Formats, and Addressing Modes. Number Systems: Binary, hexadecimal, and their applications in computer systems. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 **Total Marks:25**

ANNEXURE A
APPLICATION BASED QUESTIONS

Code of Conduct:

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode–execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.
4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Computer Architecture & Microprocessor Analysis

Comprehensive Technical Evaluation & Architectural Solutions

---

1. Smart City Surveillance System Bottlenecks & Architecture

In a modern smart city surveillance infrastructure, continuous high-definition video feeds generate massive, uninterrupted data streams. These streams must move seamlessly between camera I/O interfaces, system memory (RAM), and CPU processing cores to perform real-time automated threat detection. During peak hours, hardware resources experience heavy bus contention and memory bandwidth saturation, directly causing system lag and delayed responses.

Interaction Between CPU, Memory, and I/O

- **I/O Bottleneck & Bus Contention:** High-resolution video streams quickly consume available bus bandwidth. When multiple camera devices attempt to write frame data concurrently, severe bus contention occurs, forcing the processor to wait for bus access.
- **Memory Latency Limits:** Main memory bandwidth becomes saturated as frame buffers fill up faster than the processing threads can consume them. The mismatch between memory access latency and CPU operational clock speed creates severe memory stalls.
- **CPU Wait States:** Because data cannot be delivered into execution registers at the required throughput, CPU pipeline cycles are wasted in idle wait states rather than executing analytics algorithms.

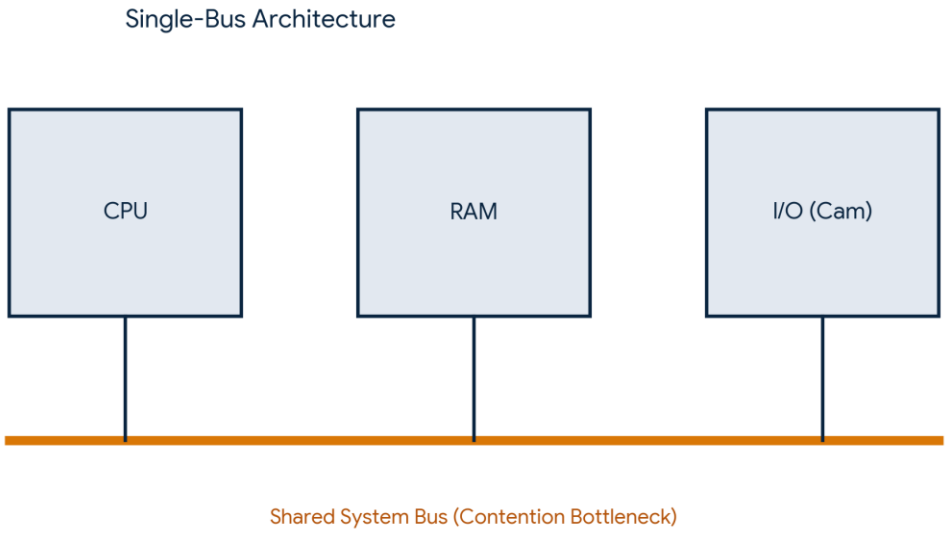

Figure 1.1: Single-bus architecture bottleneck vs shared system resources.

Bus Structures & Instruction Execution Optimization

Architectural Feature	Single-Bus Architecture	Multi-Bus Architecture
Data Path Structure	Single shared system bus for CPU, Memory, and all I/O devices.	Dedicated independent buses (CPU-Memory bus, PCIe, local I/O buses).
Throughput Capability	Low throughput; high contention during concurrent video transfers.	High throughput; decouples peripheral I/O from memory & CPU transactions.
System Performance Impact	Creates severe execution stalls and dropped video frames.	Eliminates I/O stalls, allowing continuous real-time threat analysis.

Key Execution Enhancements:

• Direct Memory Access (DMA): Allows video frames to bypass CPU execution entirely during transfers to main memory.

• SIMD / Vector Parallelism: Processes multiple image pixels in a single instruction cycle, dramatically accelerating detection algorithms.

2. Real-Time Stock Trading Processor Optimization

High-frequency real-time stock trading platforms require deterministic microsecond execution times. When processing millions of transactions per second, high Cycles Per Instruction (CPI) values lead to transaction delays and system latency. CPI degrades primarily due to execution stalls in the fetch-decode-execute cycle driven by dynamic memory lookups and branch uncertainty.

Impact of Fetch-Decode-Execute Cycle on CPI

CPU Execution Time = Instruction Count × CPI × Clock Cycle Time

1. Data Hazards & Memory Access Stalls: Order book evaluations require frequent memory lookups. Cache misses stall the execution phase for hundreds of clock cycles while data is fetched from RAM.
2. Control Hazards & Misprediction: Decision logic (e.g., condition checks on stock price thresholds) causes frequent branching. Mispredicted branches force pipeline flushes during fetch and decode stages, wasting execution throughput.

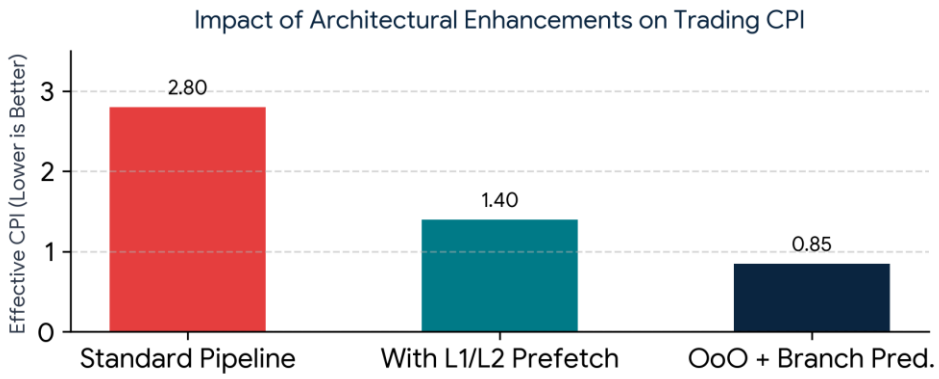

Figure 2.1: Reduction of CPI achieved through prefetching and out-of-order execution.

Architectural Enhancements to Lower CPI

- **Multi-Level Cache Prefetching:** Hardware prefetchers analyze sequential order data and preload cache lines into L1/L2 caches prior to execution, mitigating memory access stalls.
- **Out-of-Order Execution (OoOE):** Allows non-dependent instructions to execute while memory loads complete, keeping functional ALU units saturated.
- **Branch Prediction & Branchless Programming:** Employ advanced 2-bit or neural branch predictors alongside conditional instructions (e.g., CMOV) to minimize pipeline flushing.

3. 8085 Microprocessor in Industrial Temperature Control

In an 8085-based industrial temperature control system, improper data referencing and incorrect addressing mode selections lead to subtle software bugs, resulting in corrupt sensor readings and unsafe control outputs.

Addressing Modes and Error Analysis

Addressing Mode	Example Instruction	Common Cause of System Error
Direct	LDA 2050H	Hardcoding fixed addresses causes failures when memory structures or sensor buffers shift dynamically.
Register Indirect	MOV A, M	Uninitialized HL register pair points to garbage memory, corrupting control output registers.
Immediate	MVI A, 35H	Hardcoding dynamic threshold

		constants directly in code instead of configurable RAM buffers.
Register	MOV A, B	Accidental register overwrites prior to storing critical sensor readings into RAM locations.

Ensuring Reliable Operation in 8085

To ensure deterministic operation, pointers must always be explicitly initialized and sensor inputs validated before control loop execution:

```
LXI H, 2050H    ; Explicitly initialize pointer to sensor buffer
MOV A, M        ; Read sensor value into Accumulator
ANI 0FH         ; Mask higher bits to eliminate hardware noise
CPI 35H         ; Compare against target threshold value
```

4. Segmented Memory Management in 8086 Microprocessor

The 8086 microprocessor features a 16-bit internal architecture combined with a 20-bit address bus, enabling access to 1 MB of physical memory via segment registers (CS, DS, SS, ES). While segmentation allows multi-tasking and program modularity, improper segment management can cause serious runtime errors like stack overflow and data corruption.

Physical Address Calculation

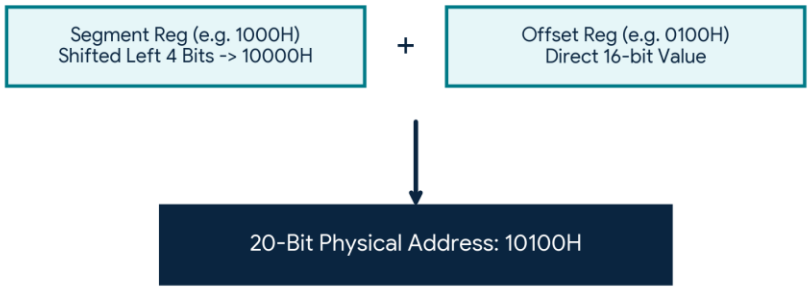

Figure 4.1: 8086 segmented memory physical address calculation mechanism.

Analysis of Failure Modes & Mitigations

- **Data Misalignment & Overlaps:** If DS is updated incorrectly, write operations will target active instruction memory in CS or shared structures in ES.

- **Stack Overflow Errors:** Because the 8086 stack grows downward toward lower physical addresses, decrements past offset 0000H wrap around to FFFFH, corrupting adjacent program data.
- **Mitigation Controls:** Explicitly initialize segment registers upon program startup and use explicit segment override prefixes (e.g., MOV AX, ES:[BX]) when operating outside default mappings.

5. Number Systems, Execution Errors & System Benchmarking

In communication networks, hexadecimal representation is widely utilized for protocol logging, instruction design, and debugging readability. However, underlying hardware execution operates purely in binary. Misinterpreting hex values leads to severe operational errors.

Hexadecimal to Binary Conversion Impact

Each hex digit maps directly to a 4-bit binary nibble (e.g., 0x7F = 0111 11112). Single-bit human conversion errors alter opcodes and data values dramatically:

- **Opcode Misinterpretation:** A mistake as small as misreading 0x78 as 0x38 changes an ADD operation into a CMP instruction, fundamentally altering program logic.
- **Endianness Errors:** Reversing byte order during manual transmission debugging (0x1234 vs 0x3412) results in incorrect pointers and memory faults.

CPU Verification & Benchmarking Frameworks

> **Hardware Controls & Benchmarking:**
> • Hardware ECC & Checksums: Implement Cyclic Redundancy Checks (CRC) and Parity Bits to validate bit integrity at the bus level.
> • Benchmarking Suites: Utilize standardized suites like EEMBC and SPEC CPU to evaluate instruction execution integrity, pipeline performance, and ALU reliability under dynamic operational loads.